

Motor Vehicle Fuel Price Regression with Deep Learning

Hoang Trung Dung*, Le Minh Kiet*, Nguyen Tat Hung*
Long Nguyen*, Van Thang*

Hanoi University of Science and Technology

*These authors contributed equally. The order of authorship is arbitrary.

January 16, 2026

Abstract

Gas prices significantly influence daily life and the broader economy, making their analysis both relevant and impactful. Numerous studies have explored this domain, yet ongoing volatility and evolving data availability continue to motivate further investigation. In this work, we apply a range of deep learning models to better understand and predict gas price behavior, drawing on concepts learned throughout our coursework. We gathered a custom dataset from 2003 to 2025 to evaluate models based on five different deep learning architectures and techniques: RNN, LSTM, GRU, LSTM+GRU hybrid, and the attention mechanism.

1 Introduction

Car and moped ownership is at an all time high in Vietnam. EVs are a viable option, but the market share for them are less than optimal currently, and is projected to remain so for the foreseeable future [source]. Aside from consumer transportation, industry also makes use of cargo trucks and ship vessels. They are the backbone of society. Cargo trucks provide supermarkets with supplies. Cargo ships allow for the globalization of goods and services. Without these means of transportation, our lives will be a lot of difficult. And what do all these vehicles have in common? They run on liquid fuel, different kinds of liquid fuel. Some run on gasoline, some diesel. The cost of fuel directly impacts our lives; it affects the price of our groceries; it affects our daily commute; it affects the growth of nations. Therefore, being able to predict the price of fuel is irrefutably an invaluable ability.

In this report, our group makes use of deep learning models to try and solve the problem of fuel price prediction. Because of the time series nature of the problem at hand, it is natural to select models with recurrent capabilities. Our group has implemented five deep learning models.

The full implementation is available in our GitHub repository¹.

2 Dataset

2.1 Raw Attributes

We obtained a set of 16 explanatory variables that were selected based on the relevant literature (for instance, [1, 4, 6]). Three of the attributes are stock markets such as Dow Jones, NASDAQ Composite, S&P 500. Others are usual rates related to United State like currency exchange rate, inflation rate, etc. The selected variables were then fed into the forecasting models through a training–testing learning process, resulting in an efficient and less error-prone models for natural gas price forecasting.

To support this modeling process, we searched for publicly available datasets from organizations in the world responsible for reporting data records of economic matters, rates like exchange rates, inflation rates, currency. In particular, we focused on two key institutions: the U.S. Energy Information Administration (EIA) and the Federal Reserve Economic Data (FRED).

¹https://github.com/KietLe2504/Project_DeepLearning_2025_1

- The U.S. Energy Information Administration (EIA) is a principal agency of the U.S. Federal Statistical System responsible for collecting, analyzing, and disseminating independent and impartial energy information. EIA provides comprehensive datasets on various aspects of the energy sector, including crude oil production, inventories, imports and exports, consumption trends, and energy prices.
- The Federal Reserve Economic Data (FRED) platform is a comprehensive online database maintained by the Federal Reserve Bank of St. Louis, offering access to hundreds of thousands of U.S. and international time series datasets. FRED covers a broad range of economic indicators, including interest rates, inflation, employment statistics, GDP figures, and commodity price indexes.

#	Name	Mean	Standard Deviation	Skewness	Kurtosis	Variance
Panel A: Oil Prices						
1	Cushing Crude Oil Prices	68.6529	22.8144	0.2348	-0.389	520.4965
Panel B: Derived Statistics						
2	5-Day Momentum	2.0863	0.9733	-0.0479	-0.4862	0.9474
3	10-Day Momentum	4.6906	1.4802	-0.0737	-0.31	2.191
4	5-Day Moving Average	68.7002	22.7436	0.239	-0.4511	517.2725
5	10-Day Moving Average	68.6871	22.7109	0.2365	-0.4709	515.7858
Panel C: Stock Indices						
6	Dow Jones Industrial Average Index	9.753	0.4865	0.308	-1.197	0.2367
7	NASDAQ Composite Index	8.416	0.7583	0.3666	-1.1841	0.575
8	S&P 500 Index	7.5842	0.5431	0.3986	-1.0268	0.2949
Panel D: Exchange Rates						
9	USD/EUR	1.2233	0.1288	0.5128	-0.4818	0.0166
10	USD/GBP	1.5279	0.2366	0.4263	-0.8392	0.056
11	JPY/USD	110.5382	17.9653	0.4789	0.4028	322.752
Panel E: Interest Rates						
12	Effective Federal Funds Rate	1.6901	1.8758	0.8928	-0.7252	3.5184
13	Bank Prime Loan Rate	4.7926	1.8586	0.9229	-0.6737	3.4545
14	1-Year Treasury Constant Maturity Rate	1.8096	1.7819	0.7498	-0.9169	3.1752
15	10-Year Treasury Constant Maturity Rate	3.042	1.1364	-0.0781	-1.0441	1.2915
16	5-Year Breakeven Inflation Rate	1.9405	0.5704	-1.261	5.3625	0.3253
17	10-Year Breakeven Inflation Rate	2.0954	0.3995	-1.2464	2.6557	0.1596

Figure 1: List of explanatory variables along with their mean, standard deviation, skewness, kurtosis, and variance as of May 2025

The information from EIA and FRED will be updated periodically and we take that through a registered API key on their website. This allows our dataset to be a rolling dataset.

Although we initially attempted to use Vietnamese datasets to achieve better local relevance, data availability proved to be a major limitation. Petrolimex, Vietnam’s largest petroleum distributor, does not provide public access to its detailed data. While PVOil does publish some data, it is limited in scope, only covering the period from 2018 onward, with approximately 200 usable data points [5]. Other potentially valuable economic and energy indicators in Vietnam are either inaccessible or not recorded systematically. As a result, we opted to use U.S. datasets due to their richness, consistency, and comprehensiveness, making them more suitable for building robust forecasting models.

2.2 Data Preprocessing

Obviously, our dataset has been provided by some organizations such as EIA, FRED, ... and have consisted of 5786 samples from 3/1/2003 up to now. For beginning, let’s see about missing value.

2.2.1 Missing values

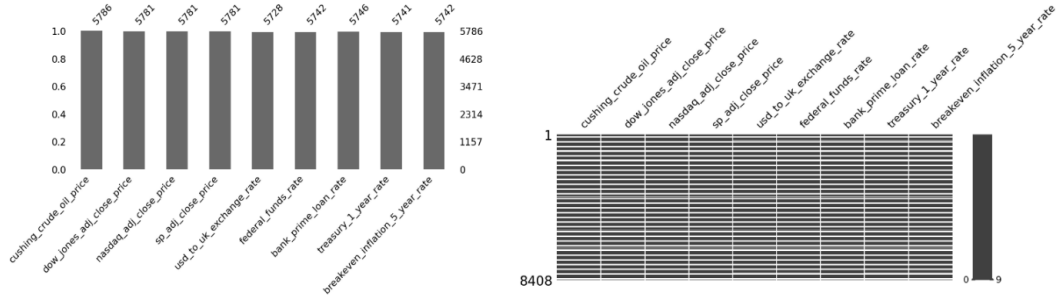


Figure 2: Missing value in our real dataset and missing value of dataset when stretches up.

From the plot, we could see that missing value consumes few percent of entire datasets. But if our dataset stretches up to date, missing values may appear periodically.

But why? The reason for this problem is there are some days that are holidays or oil prices would not be statistically record along with economic indexes. However, these values are not necessarily removed, especially economic indexes, rates such as federal funds rate, ... because of minor and rare changes in values.

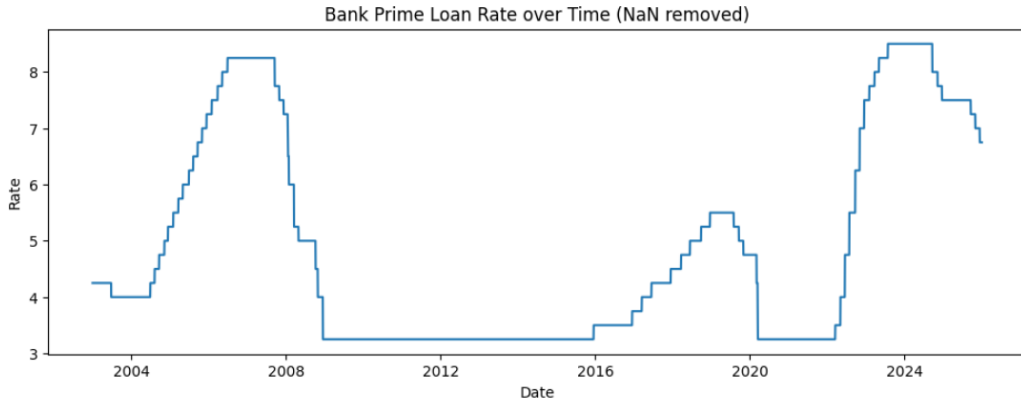


Figure 3: A plot about change in Bank Loan Rate over time. If we see in year level, rate trend varies from increase, decrease, even stable in long term. But let's see at the day level, rate remains for long term.

To deal with this problem, we have used Linear Interpolate to fill missing values. Linear interpolation is a numerical technique used to estimate missing values within the range of known data points by assuming a linear relationship between adjacent observations. It is widely applied in data preprocessing, time series analysis, and signal reconstruction when missing values occur intermittently. Linear interpolation is classified as an in-sample estimation method, not a forecasting technique.

2.2.2 Normalization

As we observe our dataset, each value varies differently in range, so we need normalizer or scale that is suitable for time-series problem like this. Therefore, Max-Min Scaling is in use. Min-Max Scaling is a normalization technique that linearly transforms numerical features to a fixed range, typically $[0, 1]$. The method preserves the relative ordering of values while rescaling magnitudes, making features comparable across different units and ranges. Given a feature x with minimum $x_{\min}$ and maximum $x_{\max}$, the scaled value x' is computed as:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (1)$$

For a general target range $[a, b]$:

$$x' = a + \frac{(x - x_{\min})(b - a)}{x_{\max} - x_{\min}} \quad (2)$$

3 Deep Learning Models

3.1 Recurrent neural network

The model implements a multi-layer RNN architecture that processes sequential data to perform one-step-ahead predictions, employing a sliding window approach to capture temporal context. The architecture is implemented in PyTorch 1.13+ and follows contemporary deep learning practices for time series analysis.

The proposed architecture employs a stacked recurrent neural network (RNN) design consisting of two primary computational blocks, as illustrated in Figure 4. The network follows the standard RNN formulation, where the hidden state at time step t is computed as:

$$\mathbf{h}_t = f(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b}), \quad (3)$$

where $\mathbf{x}_t$ denotes the input vector at time step t , $\mathbf{h}_{t-1}$ is the previous hidden state, $\mathbf{W}_x$ and $\mathbf{W}_h$ are learnable weight matrices, $\mathbf{b}$ is the bias term, and $f(\cdot)$ represents a nonlinear activation function.

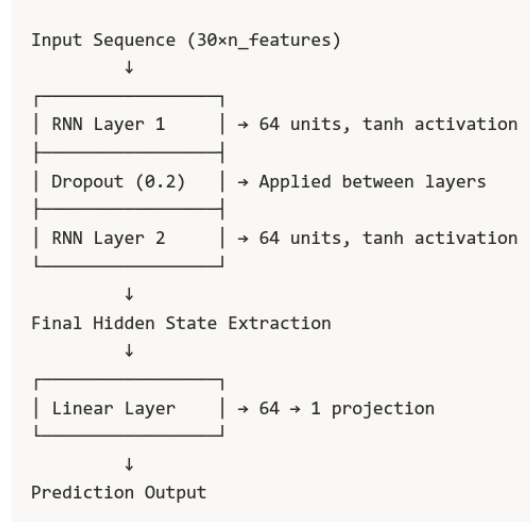


Figure 4: RNN model structure

The model comprises three parameterized layers with the configuration detailed in Table 1. The stacked RNN design enables hierarchical temporal feature extraction, while the final linear layer performs regression on the extracted temporal representations.

Table 1: Layer Configuration and Parameter Count

Layer Type	Quantity	Input Dimensions	Output Dimensions	Approximate Parameters
RNN Layer 1	1	(batch, 30, n_features)	(batch, 30, 64)	4,160*
RNN Layer 2	1	(batch, 30, 64)	(batch, 30, 64)	8,320
Linear Layer	1	64	1	65
Total	3			~12,545

*Parameter calculation assumes $n_{\text{features}} = 1$: $(1 \times 64 + 64 \times 64 + 64) = 4,160$.

The model is trained using the Adam optimizer with default momentum parameters ($\beta_1 = 0.9$, $\beta_2 = 0.999$) and a fixed learning rate of 0.001. Mean Squared Error (MSE) serves as the loss function.

Hyperparameters were selected through empirical evaluation on validation datasets, balancing model capacity with generalization performance. Table 2 summarizes the complete hyperparameter configuration.

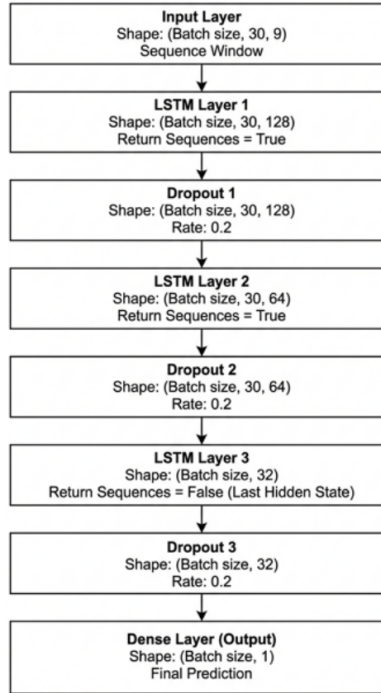
Table 2: Hyperparameter Configuration

Hyperparameter	Symbol	Value	Purpose
Training Epochs	E	50	Complete training iterations
Batch Size	B	32	Mini-batch gradient descent
Learning Rate	η	0.001	Step size for parameter updates
Hidden Dimension	d_h	64	RNN state dimensionality
Number of Layers	N_l	2	Network depth
Dropout Probability	p	0.2	Regularization strength
Sequence Length	L	30	Temporal context window
Gradient Clipping	–	Disabled	–
Weight Decay	–	0	L2 regularization

3.2 Long-short term memory

Recurrent Neural Networks (RNNs) are powerful models for sequential data, but they suffer from major limitations when dealing with long-term dependencies due to the vanishing and exploding gradient problem. When sequences become long, standard RNNs struggle to retain important information from earlier time steps.

To address this issue, Long Short-Term Memory (LSTM) was introduced. LSTM enhances the traditional RNN by adding a memory cell and a set of gating mechanisms, allowing the model to selectively remember or forget information over long periods of time. This design enables LSTM to effectively handle long-term dependencies that standard RNNs fail to capture.

**Figure 5:** The structure of our LSTM model

The proposed model architecture utilizes a deep Stacked Long Short-Term Memory (LSTM) network, specifically engineered to capture complex temporal dependencies and non-linear patterns in crude oil price fluctuations. As illustrated in Table 3, the network follows a hierarchical "funnel" design, where the number of hidden units progressively decreases ($128 \rightarrow 64 \rightarrow 32$). This structural choice facilitates the extraction of high-level abstract features while effectively filtering out stochastic noise inherent in financial time-series data.

The data flow begins with a sequence window of 30 days, incorporating 9 exogenous and endogenous features. The first two LSTM layers are configured with `return_sequences=True` to maintain the temporal dimensionality for subsequent processing. The final LSTM layer aggregates the sequence into a single context vector (Last Hidden State), which is then fed into a Dense layer for the final regression output. Dropout layers with a rate of 0.2 are

interleaved between each parameterized layer to mitigate co-adaptation of neurons and prevent overfitting.

Table 3: Detailed Layer Configuration and Data Flow

Layer Type	Input Dimension	Output Dimension	Description / Role
Input	(Batch, 30, 9)	(Batch, 30, 9)	Sequence Window ($W = 30$)
LSTM Layer 1	(Batch, 30, 9)	(Batch, 30, 128)	Temporal Feature Extraction
Dropout 1	(Batch, 30, 128)	(Batch, 30, 128)	Regularization ($p = 0.2$)
LSTM Layer 2	(Batch, 30, 128)	(Batch, 30, 64)	Intermediate Processing
Dropout 2	(Batch, 30, 64)	(Batch, 30, 64)	Regularization ($p = 0.2$)
LSTM Layer 3	(Batch, 30, 64)	(Batch, 32)	Context Aggregation (Last State)
Dropout 3	(Batch, 32)	(Batch, 32)	Regularization ($p = 0.2$)
Dense (Head)	(Batch, 32)	(Batch, 1)	Final Regression Output

Parameter Calculation: The model contains a total of **132,513** trainable parameters. The parameter count for each LSTM layer is determined by the four internal gates (Input, Forget, Cell, and Output), following the formula:

$$\text{Parameters} = 4 \times [(\text{input_dim} + \text{hidden_units}) \times \text{hidden_units} + \text{hidden_units}] \quad (4)$$

The complete set of hyperparameters used to achieve the baseline results is summarized in Table 4.

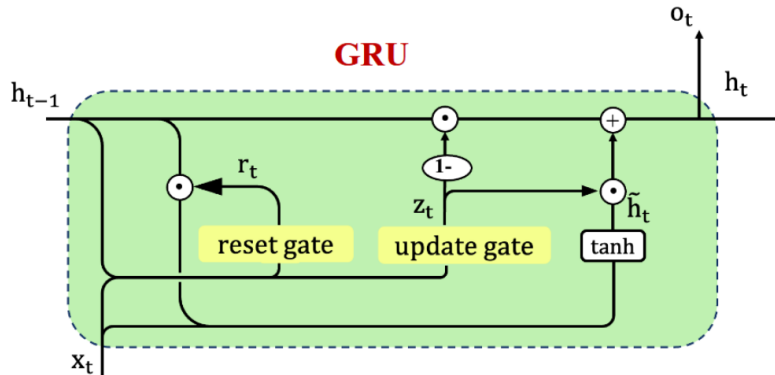
Table 4: Hyperparameter Configuration for Model Training

Hyperparameter	Symbol	Value	Purpose
Architecture	-	Stacked LSTM	Capture non-linear temporal patterns
Look-back Window	W	30	Temporal context length
Hidden Dimensions	h	[128, 64, 32]	Hierarchical feature compression
Optimizer	-	AdamW	Optimization with decoupled weight decay
Weight Decay	λ	5×10^{-4}	L2 regularization for generalization
Initial Learning Rate	η	10^{-3}	Starting step size for updates
Loss Function	L	Huber ($\delta = 1.0$)	Robustness against price outliers
Batch Size	B	32	Mini-batch stochastic gradient descent
Training Epochs	E	100	Maximum iterations (with Early Stopping)
Dropout Rate	p	0.2	Prevent neuronal co-dependency

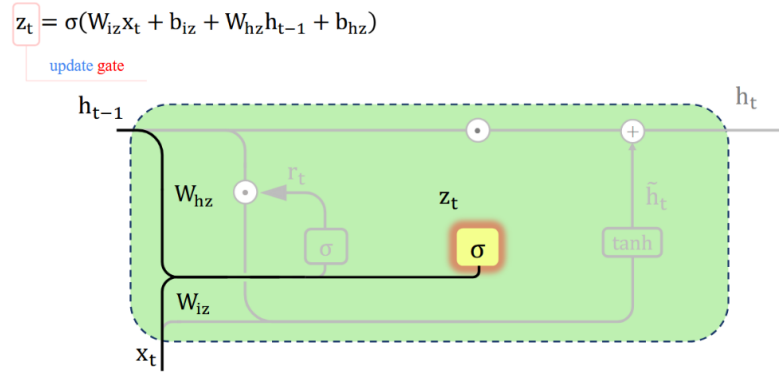
3.3 Gated recurrent unit

LSTM is a very good model that has overcome the problem of RNN when it is able of handle long term memory. But it requires a big problem of the number of parameters need to be trained. This is why GRU appear. The GRU simplifies the LSTM architecture, resulting in fewer parameters, which significantly reduces both computational overhead and training time.

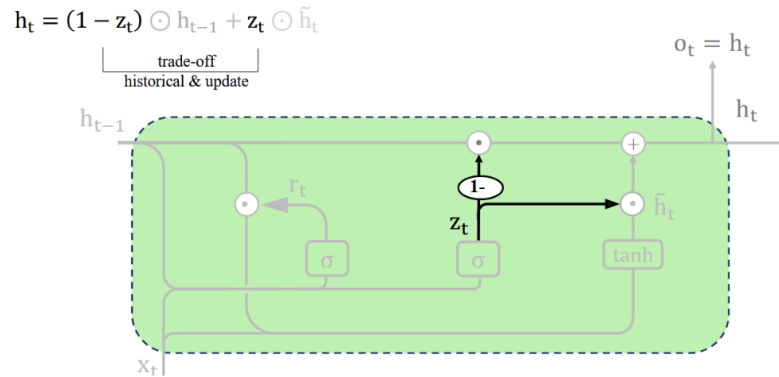
GRU simplifies the architecture as



Generally, GRU is built based on 3 main phases of LSTM: which are the forget gate, input gate, and output gate. It brings information from the past to the future. But it merges different phases, leaving only 2 primary gates, reset gate and update gate, while still perform all 3 functions. This allows the model to efficiently pass relevant information from the past into the future while maintaining a leaner structure. The reset gate r_t is responsible for deciding how much of the previous memory should be retained. It calculates the proportion of old knowledge to keep and combines it with the current input x_t to create a candidate hidden state $\tilde{h}_t$. It helps the model decide when to ignore the past while computing new content.



The update gate will now think with this new information x_t , how it should keep the historical knowledge before adding with the new memory all together to get z_t . It's not just updating the value in the output gate but also simultaneously refining the long term memory that will be passed to the next step. The update gate controls the blend between old hidden state vs. the candidate hidden state. Then combine these elements to get the predicted value h_t for the current time step.

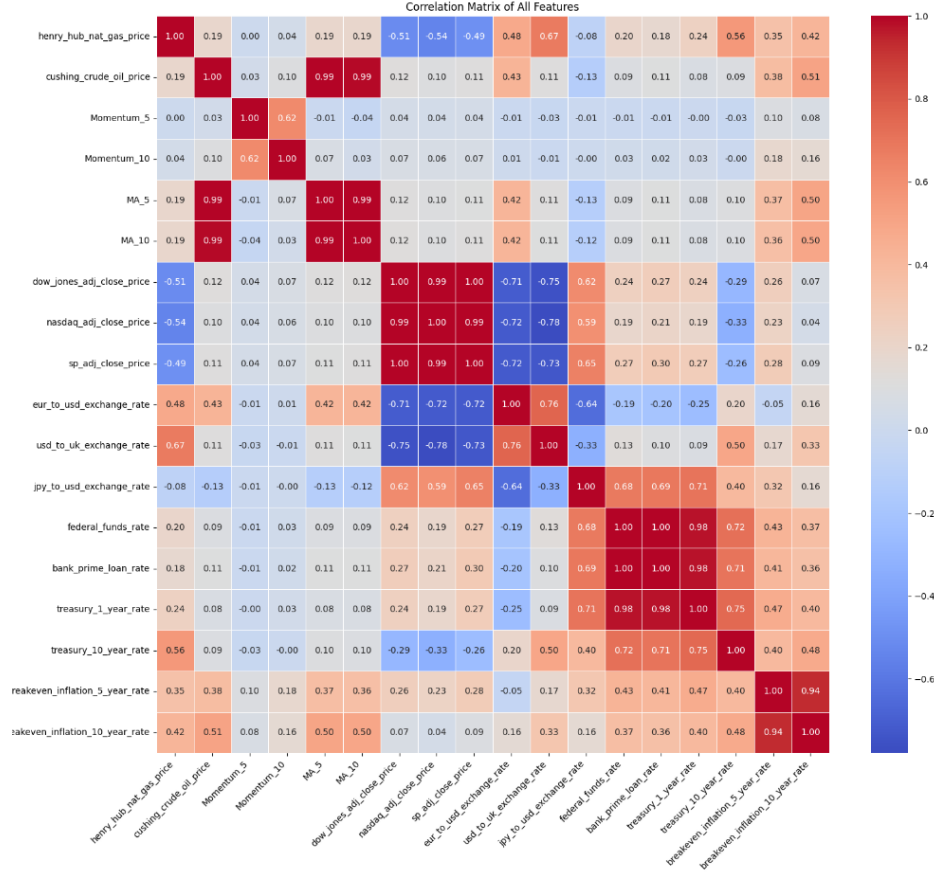


Our GRU model is built as a simple regression network with a stacked GRU core and a linear output layer. It takes in sequential data and runs it through two GRU layers with a hidden size of 32, using batch-first formatting. While the GRU produces outputs at every time step, only the final hidden state is used as a summary of the entire sequence.

That final representation is then passed into a fully connected layer to generate a single predicted value. This setup keeps the model lightweight while still being able to learn useful temporal patterns from the input data.

Training procedure

To train the model, we first using the information of the previous 30 days to predict the next day. The batch size is set to 128 and 1 hidden layer size 32 is applied. We set up this randomly to select features. A correlation matrix is drawn to choose the training features, as shown below:



We start training with those features with high correlation with the target (more than 0.3), which is `cushing_crude_oil_price`, and the performance obtained is 0.0321 (MAE), 0.0425 (RMSE), 0.9407 (R^2). Although the correlation matrix indicates a high correlation for `eur_to_usd_exchange_rate`, the data is heavily fragmented. Dropping these rows would result in significant information loss. Furthermore, attempts to fill missing values using time-series interpolation methods (such as linear or spline interpolation) proved ineffective, resulting in degraded performance in our experiments.

We also test with using all features that have relation (correlation value more than zero), the result drops down to 0.0338 (MAE), 0.0432 (RMSE), 0.9385 (R^2).

Based on this testing, we realize that the correlation matrix calculates based on the same day, which may not be suitable when we also need to take into account the information of all the things from the past.

This takes us to this feature selection: `dow_jones_adj_close_price`, `nasdaq_adj_close_price`, `sp_adj_close_price`, `usd_to_uk_exchange_rate`, `treasury_1_year_rate`, `breakeven_inflation_5_year_rate`, `bank_prime_loan_rate`, `federal_funds_rate`. These features yield high performance because they collectively capture the broader macroeconomic ecosystem that fundamentally drives crude oil demand and pricing, moving beyond simple historical price trends. Major equity indices like the Dow Jones, Nasdaq, and S&P 500 act as proxies for overall economic health and industrial sentiment, which are directly tied to energy consumption levels. Meanwhile, monetary indicators—including the Federal Funds Rate, Treasury yields, and bank prime loan rates—signal changes in the cost of capital and liquidity that influence economic expansion or contraction. Furthermore, since oil is priced globally in dollars, the USD exchange rate captures currency fluctuation impacts on affordability for foreign buyers, while inflation metrics allow the model to anticipate shifts in purchasing power. By combining these diverse financial signals, the model gains a holistic view of the market forces exerting pressure on oil prices, enabling the GRU to detect complex, non-linear dependencies that a simple univariate analysis would miss. This feature selection increases the performance up to 0.0337 (MAE), 0.0415 (RMSE), 0.9433 (R^2), letting this become the best feature group.

Adam is the preferred optimizer for this GRU model because it utilizes adaptive learning rates, meaning it auto-

matically calculates individual update speeds for every single parameter in your network rather than using one fixed speed for everything. This is crucial for financial time series data (like oil prices), which is inherently noisy and volatile; Adam’s internal momentum helps smooth out this noise to focus on the underlying trend. Additionally, GRU often suffer from unstable gradients, and Adam acts as a stabilizing force that helps the model converge faster and more reliably than standard Stochastic Gradient Descent (SGD). The hyperparameters are obtained by using grid search with the total of 490 experiments with:

hidden size $\in \{8, 16, 32, 64, 128, 256\}$

number of layer $\{1, 2, 3, 4, 5\}$

Lookback days $\in \{1, 3, 7, 14, 21, 30\}$

The learning rate found by using LRFinder is 0.007586. The optimal hyperparameters are: hidden size = 32, number of layer = 2, lookback days = 1. By using these values, GRU reaches 0.018011 (MAE), 0.024176 (RMSE), 0.981945 (R^2).

3.4 LSTM GRU hybrid model

When I discovered LSTM architecture, I was motivated about RNN Encoder-Decoder in “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation” paper written by Kyunghyun Cho et al [2]. In the paper, they proposed a novel neural network architecture that learns to encode a variable-length sequence into a fixed-length vector representation and to decode a given fixed-length vector representation back into a variable-length sequence. Therefore, based on the novelty ideal, I propose a hybrid recurrent neural network architecture that combines Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) layers through a state-transfer mechanism. The design aims to exploit the complementary strengths of LSTM and GRU for time series forecasting, particularly in scenarios involving long-term dependencies and smooth temporal dynamics. Here is an illustration of my architecture.

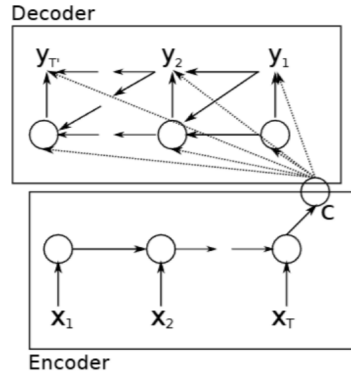


Figure 6: An illustration of the proposed RNN Encoder-Decoder in paper

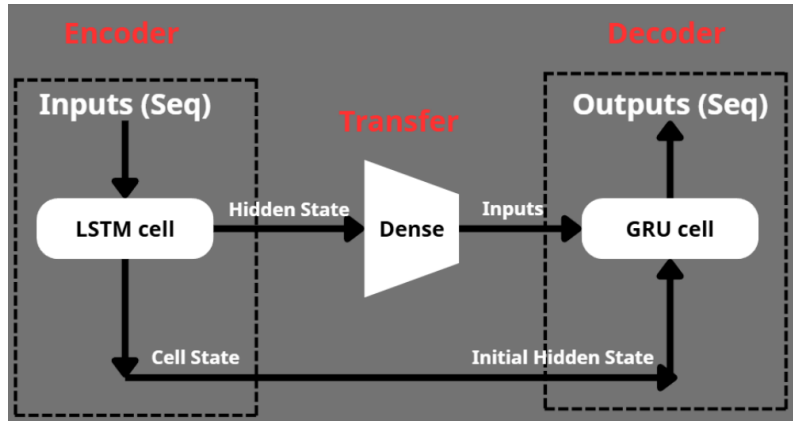


Figure 7: In my architecture, I wanted to take full advantage of both LSTM and GRU architecture. The model utilizes the LSTM layer as a temporal encoder to capture long-term dependencies present in the input time series. The GRU layer is employed as a refinement module that processes the LSTM output sequence while being initialized with transferred contextual memory.

Unlike conventional stacked architectures, where the hidden state outputs of LSTM cells are directly fed into GRU cells, the proposed hybrid structure leverages the cell state of the LSTM, which functions as a memory conveyor belt enabling long-term retention of past information. This cell state is used to initialize the hidden state of the GRU, facilitating effective knowledge transfer from the LSTM to the GRU. As a result, the GRU can learn and build upon the long-term contextual information captured by the LSTM.

Training procedure

Let's go into more detail regarding the structure and configurations. Beginning with the Encoder, at each time step t , a 9-dimensional input vector, together with the previous hidden and cell states (128 dimensions), is fed into the LSTM cell. The LSTM updates its cell state to preserve long-term temporal information while producing a hidden state at each step. After t time steps, the model outputs the final cell state and the sequence of hidden states ($128 \times t$), which jointly represent long-term memory and temporal features for downstream processing.

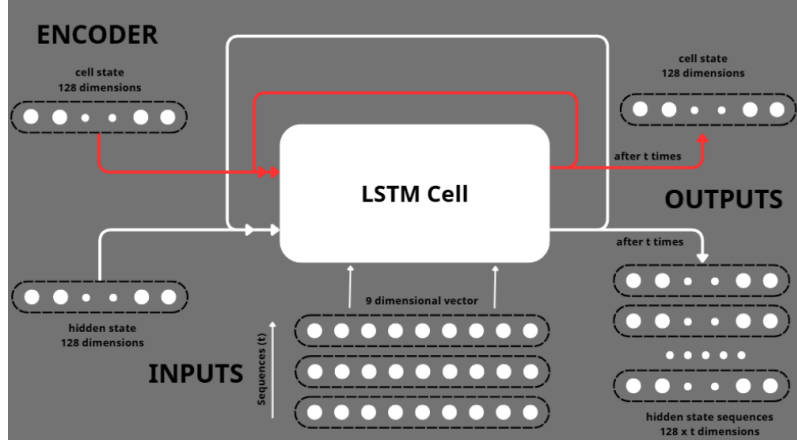


Figure 8: Main architecture in Encoder side.

The final cell state output of the LSTM (128 dimensions) is first projected through a fully connected transformation followed by a $\tanh$ activation to match the GRU hidden-state dimensionality (96 dimensions). This transformed vector is used to initialize the GRU hidden state. In parallel, the LSTM hidden-state sequence ($128 \times t$) is directly transferred as the input sequence to the GRU, enabling the GRU to learn from both long-term memory and temporal features extracted by the LSTM.

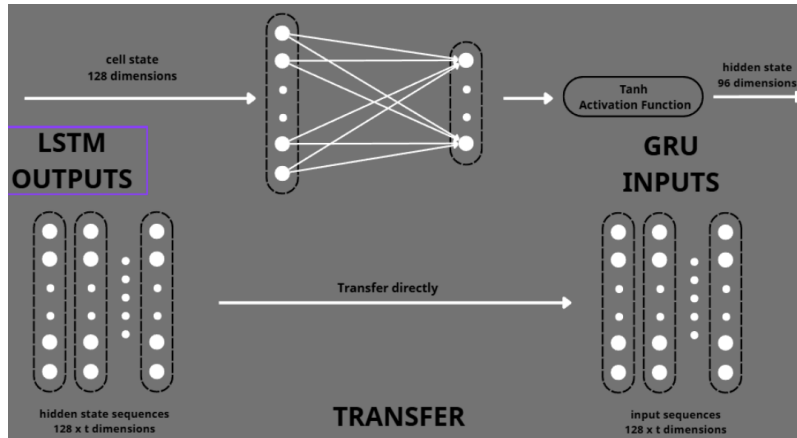


Figure 9: Transfer process.

The GRU decoder is initialized with a 96-dimensional hidden state transferred from the encoder. At each time step, the GRU cell receives the input sequence and updates its hidden state accordingly. After processing t time steps, the final hidden state is passed through a fully connected layer to generate the predicted output values. This decoding process transforms the learned temporal representations into the target price predictions.

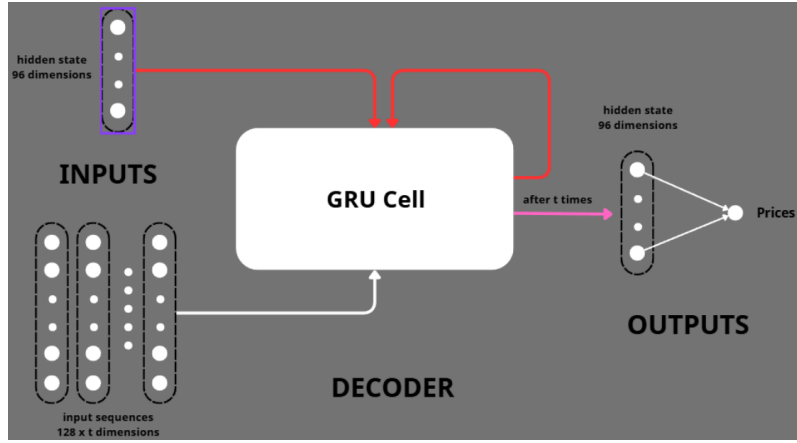


Figure 10: Main architecture in Decoder side.

Moreover, a dropout rate of 0.1 is applied to mitigate overfitting. The model is trained using the Adam optimizer with a learning rate of 0.001, and Mean Absolute Error (MAE) is employed as the loss function due to its reduced sensitivity to outliers. The training procedure runs for 50 epochs with a batch size of 15, incorporating early stopping when no improvement in model weights is observed. All of these configurations result from repeated experimentation and systematic hyperparameter tuning rather than arbitrary or heuristic choices. Finally, the trained models are used to predict daily and weekly oil prices, employing lookback windows of 5 and 20 days, based on 6,722 and 6,071 samples, respectively.

Model: "functional_3"

Layer (type)	Output Shape	Param #	Connected to
input_layer_3 (InputLayer)	(None, 5, 9)	0	-
lstm_3 (LSTM)	[(None, 5, 128), (None, 128), (None, 128)]	70,656	input_layer_3[0]...
dense_6 (Dense)	(None, 96)	12,384	lstm_3[0][2]
gru_3 (GRU)	(None, 96)	65,088	lstm_3[0][0], dense_6[0][0]
dense_7 (Dense)	(None, 1)	97	gru_3[0][0]

Total params: 444,677 (1.70 MB)
 Trainable params: 148,225 (579.00 KB)
 Non-trainable params: 0 (0.00 B)
 Optimizer params: 296,452 (1.13 MB)

Figure 11: Here are number params in our models. We can see that our model has totally 148,225 params and can be considered lightweight. But we still need to check ability of Models (please see in Result part)

3.5 Attention-based model

Of course, we feel obligated to implement a model that uses the attention mechanism as it is the poster child of modern deep learning. Our model uses a textbook implementation [3] of the transformer encoder and a rather simplistic “decoder” to produce the final output. We built the model from scratch using Keras layers.

Regarding the model architecture, as can be seen in Figure 12, our model consists of two transformer encoder blocks connected in series and a simple decoder block that is actually just a slicing layer that extracts the last time step, and a dense layer that maps the last time step to a single value for gas price prediction. We also have dropout layers after each attention block and each feedforward layer. These dropout layers are only active during training, of course.

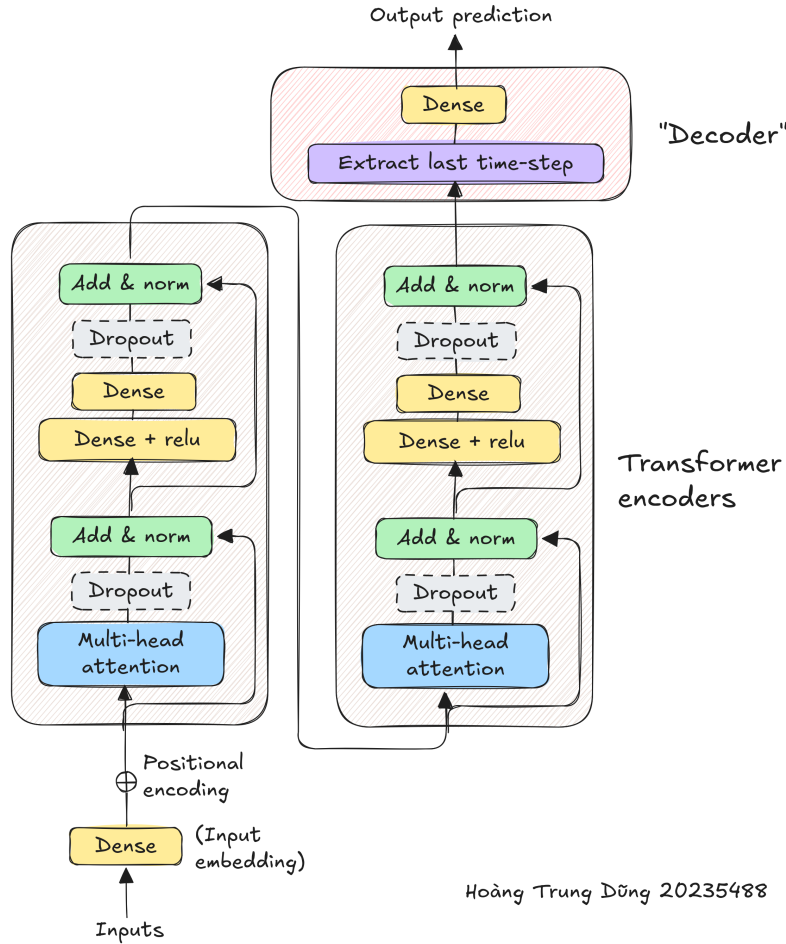


Figure 12: A very nice visualization of our attention-based model. The inputs are fed into an input embedding layer, then two consecutive transformer encoders, and finally the last time-step is extracted and fed into a dense layer to reduce it to a single value for the gas price prediction.

But enough about the general architecture of the model. An architecture is no more useful than its set of well-tuned hyper-parameters. For our given dataset, which has 9 input variables, it was determined that each multi-attention block shall have 4 heads and each head shall work with 64 dimensions. The feedforward layer with relu in the encoder operates with 128 dimensions, and the dense layer right after that reduces it back to 64 dimensions. The dropout rate is set at a reasonable 0.2, meaning 20% of weights will be randomly set to zero each run, preventing overfitting.

The model is trained with the Adam optimizer with MSE as the metric. Callbacks have also been set up to halve the learning rate if model performance doesn't improve after 5 epochs, and to stop training entirely if there's no improvement after 10 epochs.

With these hyper-parameters and architecture, the model seems to fit the data nicely and produces acceptable results. For more details on how these parameters were acquired and why some hyper-parameters are set to this value and not another, refer to the [Appendix A](#).

4 Results

4.1 RNN model performance

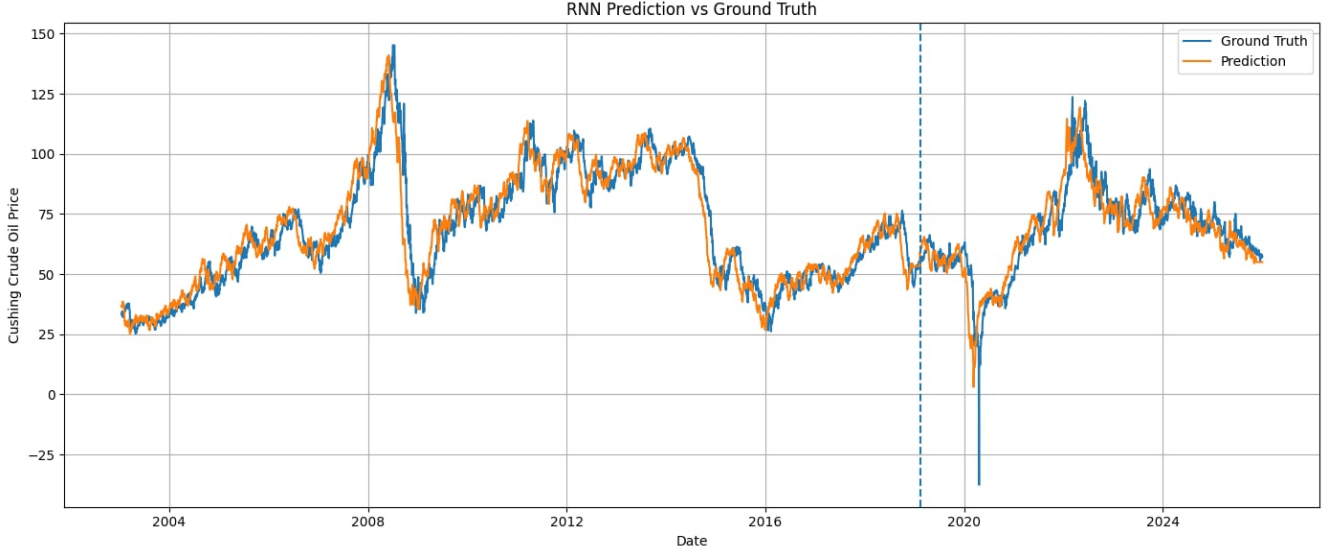


Figure 13: The performance of our RNN model on the train set and test set

4.2 LSTM model performance

The LSTM model obtained an RMSE of 0.0404 and MAE of 0.0275 on the training set, while maintaining low generalization errors on the test set with an RMSE of 0.0643 and MAE of 0.0541. In terms of goodness of fit, the model reached a high coefficient of determination, with an R^2 score of 0.9867 for training and 0.8199 for testing. These results indicate that the proposed LSTM model is able to capture the underlying temporal patterns of crude oil prices while preserving strong generalization capability on unseen data.

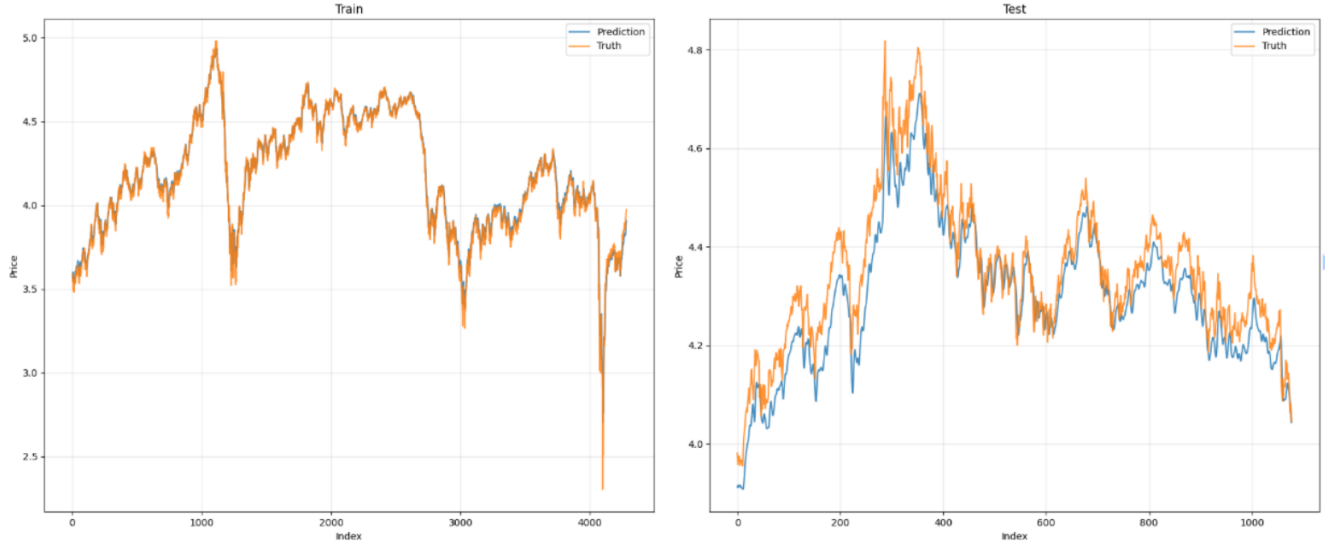


Figure 14: The performance of our LSTM model on the train set and test set

4.3 GRU model performance

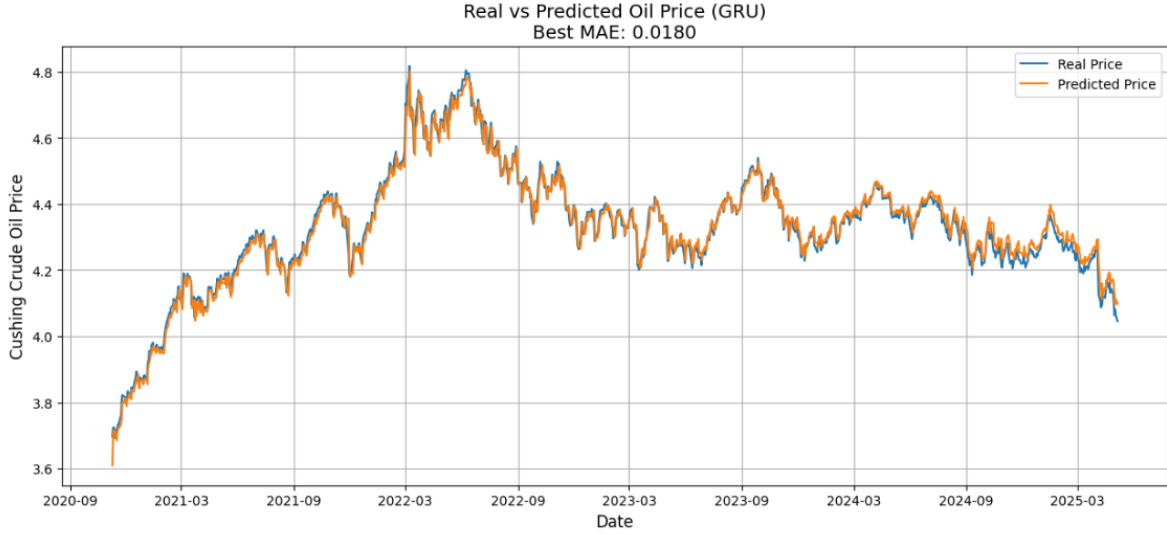


Figure 15: The performance of our GRU model on the train set and test set

The chart demonstrates the exceptional accuracy of the GRU model in forecasting Cushing Crude Oil prices. The most significant indicator of this precision is the extremely tight alignment between the predicted values (orange line) and the real prices (blue line) throughout the entire time series. This strong correlation is quantified by the low Mean Absolute Error (MAE) of 0.0180, which suggests that the average deviation between the forecast and the actual data is negligible. Visually, the model successfully mirrors the complex volatility of the market, capturing both the sharp peak of approximately 4.8 in early 2022 and the subsequent erratic fluctuations without significant lag or divergence. This consistent overlap indicates that the model is highly robust and effective at learning the temporal patterns of the dataset.

Finally, to validate the robustness of our feature selection, we re-evaluated the candidate feature groups using the newly identified optimal hyperparameters. Remarkably, the results remained consistent with our initial findings. The feature group comprising major equity indices (Dow Jones, Nasdaq, S&P 500) and monetary indicators continued to outperform other combinations.

This consistency confirms that the predictive power of these selected features is intrinsic to the market dynamics driving oil prices, rather than being an artifact of a specific model configuration. It reinforces our conclusion that while the immediate historical price (Lookback = 1) provides the baseline signal, the broader macroeconomic indicators provide critical context for correcting and refining the forecast.

4.4 LSTM+GRU hybrid model performance

Firstly, we trained models to predict future oil prices, with lookback time is five previous days before predict day. Below is the performance of the model on the training set and testing set.

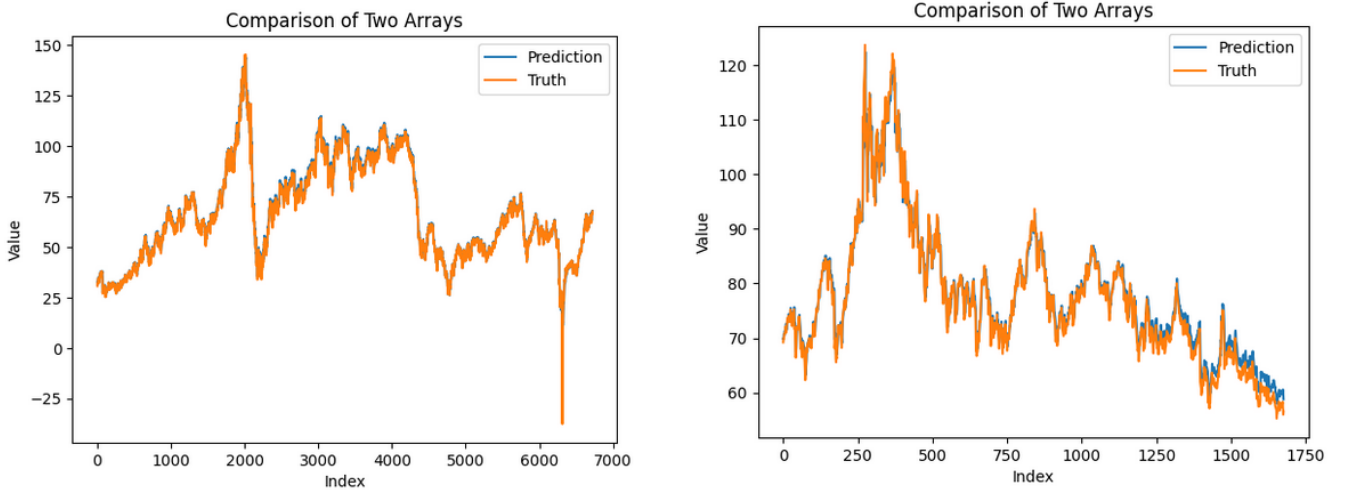


Figure 16: Comparison between model predictions and ground-truth values on the training set (left) and the test set (right). The predicted series closely follows the true observations

The figure illustrates the comparison between predicted values and ground-truth observations on the training dataset. The predicted series closely overlaps with the true values across most time steps, indicating that the model fits the training data well and successfully captures the underlying temporal dynamics. The model accurately follows both gradual trends and short-term fluctuations. Minor discrepancies are observed during periods of abrupt changes and extreme values; however, these deviations are limited and do not significantly affect the overall pattern. This result suggests that the proposed model exhibits strong learning capability without severe underfitting on the training data.

The comparison plot between the model's predictions and ground truth values on the test set reveals a strong initial alignment, with predictions closely tracking the truth up to approximately index 500. Beyond this point, the predictions increasingly underestimate the true values, particularly in the later indices (around 1000–1750), where divergence becomes more pronounced. This suggests the model captures overall trends well but may suffer from cumulative errors or overfitting issues in prolonged sequences, warranting further investigation into potential biases or regularization techniques.

Moreover, we also investigated the weekly oil price model and inspected its performance.

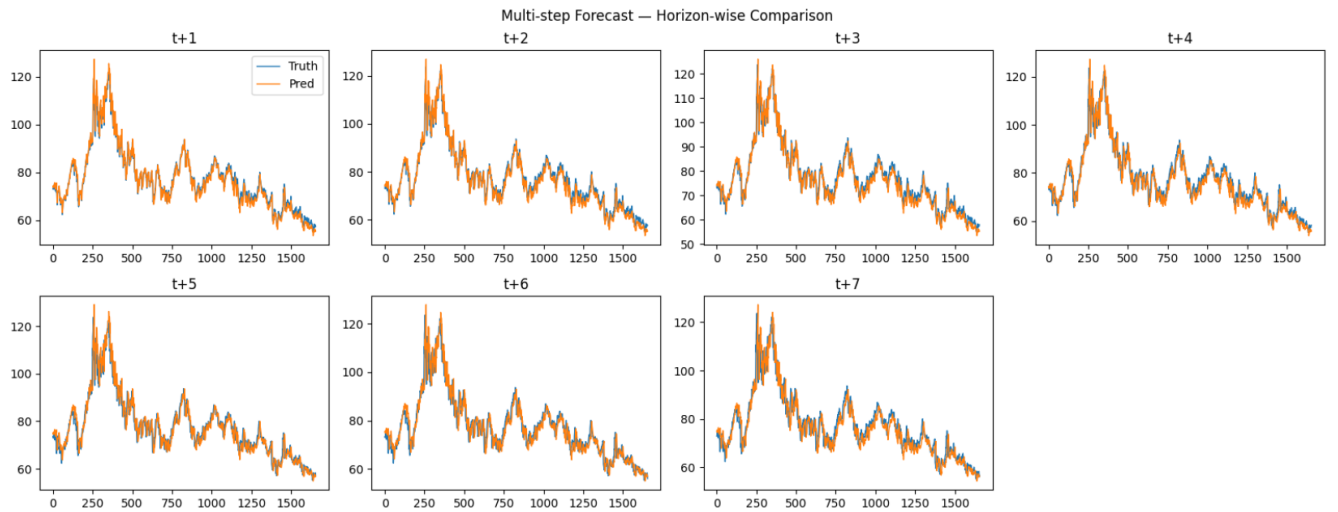


Figure 17: Performance of weekly oil prediction model.

4.5 Attention-based model performance

Our attention-based model performs really well on this small dataset, with RMSE scores of 0.0233 on the test set and 0.03255 on the train set:

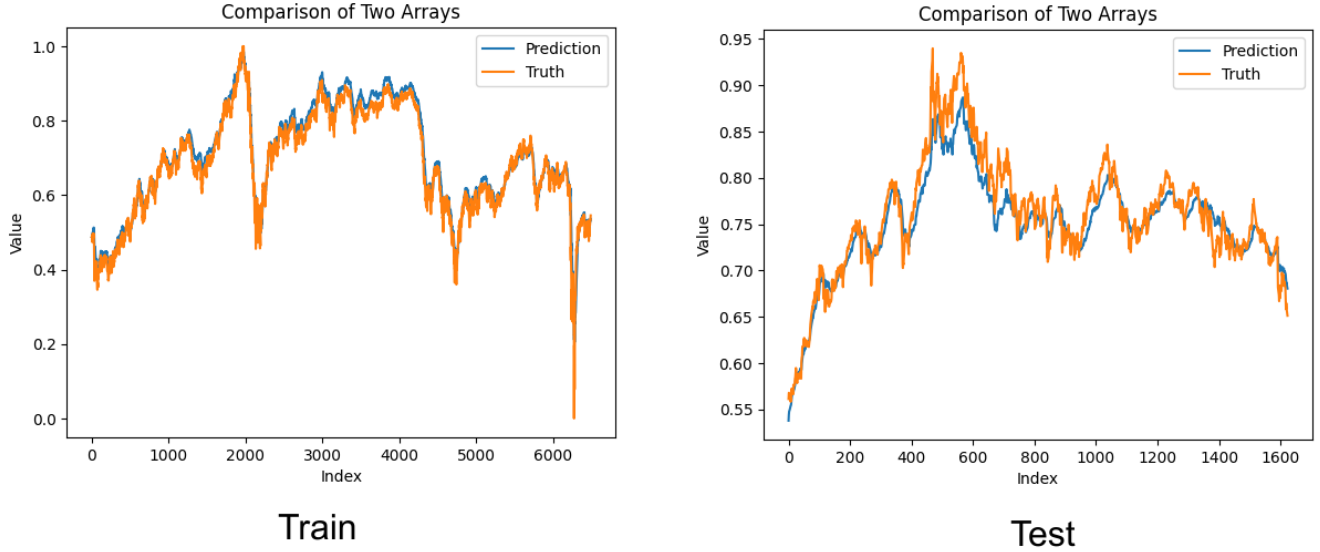


Figure 18: The performance of our attention-based model on the train set and test set

As can be seen, the model seems to fit the train set a bit better than the test set. But in general the model has successfully captured the main trends of the price points in the test set. The model does seem to exhibit some hedging when it comes to the higher peaks in the test set, but does pretty well in capturing the more moderate trends in the second half the graph.

References

- [1] Ervin Ceperic. Short-term forecasting of gas prices using machine learning and feature selection algorithms. *Energy*, 140:893–900, 2017.
- [2] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder–decoder for statistical machine translation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014. arXiv:1406.1078v3.
- [3] Jeff Heaton. Applications of deep neural networks, 2020.
- [4] Vinod Mishra and Russell Smyth. Are gas spot and futures prices predictable? *Economic Modelling*, 54:178–186, 2016.
- [5] PetroVietnam Oil Corporation (PVOIL). Gia xang dau, 2025. Accessed May 2025.
- [6] Moting Su, Zongyi Zhang, Ye Zhu, Donglan Zha, and Wenying Wen. Data driven gas spot price prediction models using machine learning methods. *Energies*, 12(9):1680, 2019.

A Attention-based model hyper-parameter experiments

A.1 Experiments with the number of encoder layers

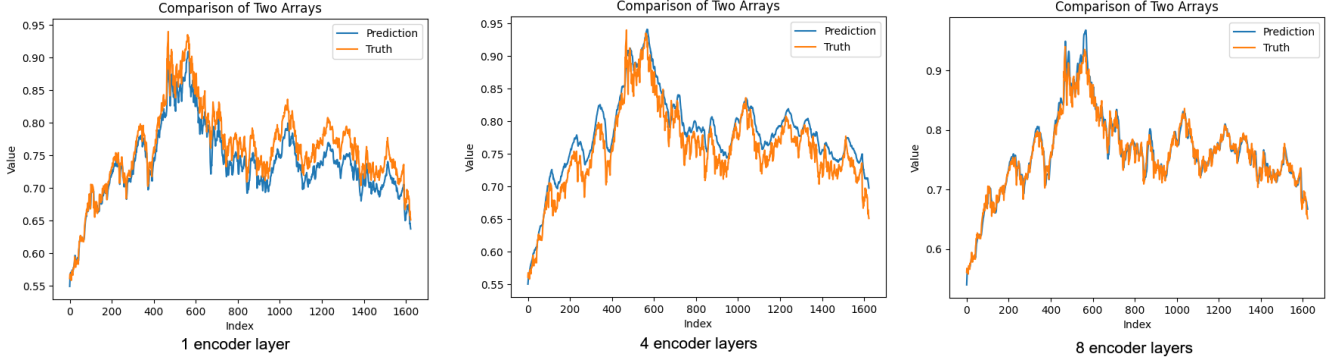


Figure 19: Comparison of model performance in the test set with 1, 4 and 8 encoder layers

With only 1 layer, the model overfits and comes up with values smaller than the target values. With 4 layers, an interesting phenomenon happens where the model appears to "hedge" and produces values that are more smoothed out. With 8 layers, it is clear that the model performs really well, but at the cost of having 4 times the number of encoders as the original model. This is a lot more expensive to train and deemed not worth it for something as volatile as gas price. We consider the 2-layer model to be sufficient at producing useful information regarding gas price.

A.2 Experiments with model dimension and feedforward dimension

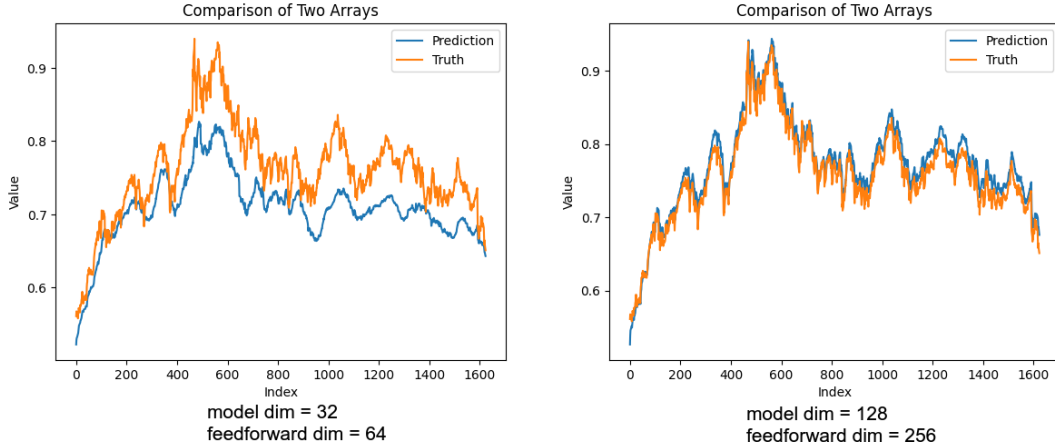


Figure 20: Comparison of model performance in the test set with half and twice the dimensions

With the model dimension halved to just 32, and the dimension of the feedforward layer reduced to just 64, the model underfits the target and smooths out a lot of details. With twice the dimensions of the original model, however, the model has more than enough capacity to learn the pattern and produce a tighter fit. However, the added cost of training the model is deemed not worth it for the return.